SkillUp

In collaboration with
IBM **SkillsBuild**

AI Racing League · Session 2 of 4

Python for racers: your first Bob-built upgrade

Read the driver code with Bob, build a tuning card, then plan, build, race and log one upgrade.

Thursday 1 October 2026
3:00 – 4:30 PM GMT · 90 minutes · hands-on, follow along



Today: from reading the code to racing your upgrade

0:00	Welcome back, Session 1 recap, baselines	Talk
0:05	Python for racers, the pit-wall method, reading a diff	Talk
0:17	Part 1 · Read the driver: systems check, map the code, tuning card, predict, safe copy	Follow along
0:45	Pit-wall briefing: choose your upgrade	Talk
0:48	Part 2 · First upgrade: plan, build, race, diagnose, changelog	Follow along
1:22	Before Session 3, preview, questions	Wrap



20 min talk · 62 min hands-on with IBM Bob · 8 min wrap

Where we are

Tue 29 Sep

AI at work, meet the car

Done: Bob on everyday jobs, baseline lap, Bob explained the code.

Thu 1 Oct · today

Python for racers

Read the driver, tuning card, safe copy, first one-change upgrade raced and logged.

Tue 6 Oct

Engineer the corners

Plan mode for braking and corner speed, track sensors, debugging crashes.

Thu 8 Oct

Race ready

Zero-damage runs, documentation, submission prep.

> Today's finish line: every learner or team leaves with v1 raced, logged, and written up in a changelog.

From Session 1, you already know

Bob

Three modes

Ask to understand, Plan to decide, Code to change. Auto-approval stays off.

Prompts

Four parts

Scope, goal, shape and guardrails. "Don't edit yet" is a guardrail.

The car

Sense, decide, act

Sensors in, Python rules decide, actions out. We improve the rules.

> In chat now: your baseline lap time and damage. No baseline yet? Write "none": you'll use the reference time today.

Two labelled boxes: S in, R out

S · what the car senses

S['speedX']	Speed along the track
S['trackPos']	Left or right of centre
S['angle']	Angle to the track direction
S['track']	19 distances to the edge ahead

R · what the car does

R['steer']	−1 (right) to 1 (left)
R['accel']	0 to 1 throttle
R['brake']	0 to 1 brake
R['gear']	−1 to 6

In Python these are called dictionaries: a box of values, each with a name in quotes. The driver reads S and writes R, many times a second.

The driver, in eight lines

```
target_speed = 100                          # the speed it aims for
R['steer'] = S['angle'] * 10 / PI           # point along the track
R['steer'] -= S['trackPos'] * 0.10          # drift back to centre
if S['speedX'] < target_speed:              # too slow?
   R['accel'] += 0.01                       #   a little more throttle
else:                                       # too fast:
   R['accel'] -= 0.01                       #   a little less
if S['speedX'] > 50: R['gear'] = 2          # change up
```

Steering
two lines: angle and centring

Throttle
tiny steps towards one target speed

Gears
change up at fixed speeds

Missing
no braking, no look-ahead

Simplified from the standard snakeoil3 example driver. Your kit's values may differ: your tuning card in B2 will show the real ones.

Three kinds of problem you'll meet today

Syntax

Python won't run

IndentationError: a line isn't lined up with the lines around it.

Logical

Runs, but drives badly

The car is slower, or runs wide at the downhill corner.

Functional

Something is missing

The car never brakes, because no line tells it to.

Same three types as the Granite troubleshooting module. In B8, Bob tells you which one you hit.

The pit-wall method

1. **Copy**
   snakeoil3_gym.py is never edited
2. **Predict**
   write what you expect first
3. **One change**
   so you know what moved the time
4. **Review**
   read the diff before approving
5. **Race and log**
   time, damage, keep or revert

> **Golden rule:** a clean slower lap beats a fast lap with damage. Damage above zero means the lap doesn't count.

Reading a diff

```
- target_speed = 100

+ target_speed = 90 + S['track'][9] * 0.5   # BOB-v1
```

Illustration only, not a recommended value

- [ ] It edits snakeoil3_v1.py, not the original
- [ ] Only a few lines are green
- [ ] Every changed line has # BOB-v1
- [ ] Connection and port code untouched
- [ ] You can explain each green line

Red is removed, green is added. Reject is always safe: nothing changes until you approve.

Part 1 of 2

Read the driver

Systems check, then Bob maps the code, builds your tuning card, predicts a change, and makes a safe copy you race once.

Follow along

Every prompt is on your learner sheet

Stuck?

Pit crew in chat, keep going with the code

Checkpoint

Tuning card and a raced copy

SkillUp

In collaboration with IBM **SkillsBuild**

Systems check

Ask mode

I'm trying to run snakeoil3_gym.py with TORCS and got this error:
[paste the error]
Explain it in plain English and give me the fix step by step. Don't change any files.

Skill

Making sure everyone can race today, even if Session 1 setup isn't finished.

Stage

Setup

1 · Map the driver

Ask mode

Open @snakeoil3_gym.py and find the function that makes the driving decisions.
1. Give me its name and line number.
2. List every sensor it reads (the S values) and every action it sets (the R values), with one plain-English sentence each.
3. Show me the 5 most important lines and explain each like I've never coded.
Do not change any code.

Skill

Using Bob to find and explain the part of the code that matters, without changing it.

Stage

Discovery

2 · Build a tuning card

Code mode

Create tuning-card.md, a one-page tuning card for the driving function in @snakeoil3_gym.py.
Make a table with: setting, line number, current value, what it controls, what happens if I raise it, what happens if I lower it, crash risk (low / medium / high).
Only create tuning-card.md. Do not edit snakeoil3_gym.py.

Skill

Getting Bob to turn code into a reference document you'll use for the rest of the challenge.

Stage

Discovery + document

3 · Predict before you change

Ask mode

Using @tuning-card.md, predict what would happen on Corkscrew if I [describe one change, e.g. raised the target speed by 20%].
Give me: the likely effect on lap time, the likely effect on damage, and which part of the track would be most affected.
Be honest about how uncertain this is.

Skill

Forming a hypothesis before changing code, so every test teaches you something.

Stage

Planning

4 · Safe copy and control run

Code mode

```
Make an exact copy of @snakeoil3_gym.py called snakeoil3_v1.py.
Do not change anything inside it.
Confirm the two files are identical.
```

1. TORCS: New Race → blue screen
2. python snakeoil3_v1.py
3. Compare with baseline: small differences are noise

Skill

Protecting your baseline and checking that results are repeatable before you test anything.

Stage

Build

Choose your first upgrade

Quicker throttle

Risk: Low

The car takes a long time to reach its target speed. Let it press the throttle harder when it's well below target.

Watch for: Wheelspin out of slow corners

Calmer steering

Risk: Low–medium

If the car weaves or runs wide, adjust how strongly it steers back towards the centre of the track.

Watch for: Too calm and it runs off on tight corners

Look-ahead speed

Risk: Medium

Set the target speed from how far the track goes straight ahead: a long view means go faster, a short view means slow down.

Watch for: Too greedy on the straights, late for the corner

Brake before corners

Risk: Medium–high

When the road ahead is short and the speed is high, brake instead of only lifting off the throttle.

Watch for: Braking too hard or too early costs time

Already have an idea from Session 1? Use it. Never coded? Start with a low-risk upgrade.

Part 2 of 2

Your first Bob-built upgrade

Plan one change, let Bob build it in your copy, review the diff, race it, diagnose what happened, and write it up.

Follow along
Same chat, switch modes as we go

Stuck?
Race what you have, log it, ask in chat

Checkpoint
v1 in your lap log and changelog

5 · Plan your upgrade

Plan mode

I want to build v1 of my race car driver in @snakeoil3_v1.py.
My idea: [your idea from Session 1, or one from the starter upgrade menu].
Plan the smallest possible change that tests this idea: which lines change, the new logic in plain English, the expected effect on lap time and damage, and what could go wrong.
One change only. Don't write code yet.

Skill

Turning an idea into the smallest possible change before any code is written.

Stage

Planning

6 · Build v1 and review it

Code mode

```
Implement the plan in @snakeoil3_v1.py only.
Change as few lines as possible and keep everything else identical.
Mark each changed line with the comment  # BOB-v1
Then list the changed lines and explain each in one sentence a beginner
would understand.
```

Skill

Letting Bob write the code while you review every change before it's saved.

Stage

Build

7 · Race v1 and log it

1. TORCS: New Race → blue screen
2. python snakeoil3_v1.py
3. Watch the part of the track you predicted would change
4. Log time, damage, top speed
5. Keep it, or fix it in step 8

Version	Lap time	Damage	Keep?
base	__:__.__	___	Yes
copy	__:__.__	___	—
v1			
v1.1			

> Faster with damage doesn't count. Slower but clean is still a result: it tells you something.

8 · Diagnose and iterate

Ask mode

I ran snakeoil3_v1.py on Corkscrew.
Baseline: [time, damage]. v1: [time, damage].
What I saw: [e.g. spun at the first downhill corner].
Terminal output: [paste any error, or write none].
Is this a syntax, logical or functional problem? Explain the most likely cause in plain English. Don't change any files.

Then · Code

Make the smallest change in @snakeoil3_v1.py that fixes this.
Mark it # BOB-v1.1 and tell me what to watch for on the next lap.

Skill

Classifying a problem (syntax, logical, functional), getting a plain-English diagnosis, and fixing it with the smallest change.

Stage

Validate and improve

9 · Write your changelog

Code mode

Create CHANGELOG.md, or add to it if it already exists, with an entry for each version I raced today.
For each: version, what changed and why, lap time and damage before and after, and whether I kept it.
Here is my lap log:
[paste your lap log rows]

Skill

Documenting each version as you go, so your final submission explains itself.

Stage

Document and deliver

Post your best clean lap

Best time with zero damage

In chat now

Name or team, version, lap time, damage

On Discord

Same details, plus one line on what you changed

Not faster yet?

Post your clean v1 anyway. Session 3 is where the big gains are.

Before Session 3 on Tuesday

- [ ] Keep progressing your mandatory learning plan activities (individually)
- [ ] Run at least two more one-change upgrades: predict, build, race, log
- [ ] Keep damage at zero and update CHANGELOG.md after every race
- [ ] Note which corner of Corkscrew costs you the most time
- [ ] Post your best clean time on Discord

Next time: engineer the corners

Plan mode

A corner strategy

Bob plans when to brake and how fast to take each corner, using the track sensors.

Sensors

Seeing ahead

The 19 distance rays tell the car a corner is coming before it arrives.

Debugging

Crashes, explained

Diagnose spins and wall hits with Bob, and fix them without losing speed.

Pit crew and links

Community and leaderboard (Zoom)

https://us04web.zoom.us/launch/chat/v2/eyJzaWQiOiJkNTVhOTI2Y2Y5MDI0OTE0ODA2OTE4NjQ2ZDliY2RhOEBjb25mZXJlbmNlLnhtcHAuem9vbS51cyJ9

Learning plan

https://skills.yourlearning.ibm.com/activity/PLAN-6FA860C53B3A

TORCS and Python scripts

https://ibm.box.com/s/sud4g3o1mqyfqoa4ab9lr80vh8zc1ri6

Get IBM Bob

https://ibm.biz/IBMBoBUniversity

Baseline lap video

https://youtu.be/xxVCPDxlxng

Bob updates the code (video)

https://youtu.be/RujPYbBMp48

Project submission: SkillUp support email **[add address]** by Sunday 11 October.

Feedback Link and QR Code

https://survey.zohopublic.in/zs/eqCGL3



Copyright Skill-Up technologies, All rights reserved . Submitted for evaluation purpose only as per existing NDA

SkillUp

In collaboration with

IBM **SkillsBuild**

SkillUp

In collaboration with

IBM **SkillsBuild**



Thank you

For further queries, email us at
ibmskillsbuild.emea@skillup.online